

**ROWSense: AN AI-ENABLED UNMANNED GROUND
VEHICLE (UGV) FOR UNDER-CANOPY CROP SCOUTING**

BS ELECTRICAL ENGINEERING

Session: 2025 - 2026

By

Huzaiifa Saeed

Roll No: 26100070

Mohammad Junaid

Roll No: 26100391

Mohid Joya

Roll No: 26100141

Virad Munir

Roll No: 26100285

Advisor:

Dr Hassan Jaleel,
Assistant Professor, EE.



**DEPARTMENT OF ELECTRICAL ENGINEERING,
SYED BABAR ALI SCHOOL OF SCIENCE AND
ENGINEERING,**

**LAHORE UNIVERSITY OF MANAGEMENT SCIENCES,
PAKISTAN**

PROJECT ABSTRACT

With the passage of time, resource availability has become scarce, and farms need smarter ways to grow more with less. Irrigation precision to conserve water, responsible pesticides application to cut chemical waste, and operations optimization to increase overall crop yield has become even more significant. Aerial platforms such as Unmanned Aerial Vehicles (UAVs) equipped with advanced imaging techniques can plot vegetation maps, count plant tillers, and measure valuable agronomic indicators such as Leaf Area Index (LAI). However, foliage occlusion and rotor downwash reduce the reliability of close-range detail when fine texture matters, making it harder to detect under-canopy symptoms early.

Our project RowSense addresses this gap with a compact Unmanned Ground Vehicle (UGV) that drives between crop rows under the canopy, acquires near-canopy imagery, and runs an on-edge computer-vision pipeline during a pass. The rover combines a custom four-wheel-drive chassis with an Intel RealSense D435 depth camera, a WheelTec N100 IMU, and a Raspberry Pi 5 for on-board real-time processing. Autonomous navigation is implemented through a behaviour-based stack composed of IMU-based heading hold for straight-line traversal between rows, depth-based obstacle avoidance using a path-frame transform that anchors a corridor mask to the intended trajectory, and a vision-based row-end turning module that uses orange cone markers as headland landmarks. A YOLOv26 object-detection model is trained on a custom under-canopy wheat dataset to flag wheat heads, tillers, and stripe rust as the rover drives between beds.

Validation was carried out on the LUMS wheat plots over an 11-week growing cycle of the Galaxy variety. A total of 1163 annotated images, expanded to 8265 samples after augmentation, were used to train YOLOv26, which achieved 97% accuracy on wheat stripe rust, 76% on wheat heads, and 74% on tillers. Forty field trials each were run for the obstacle-avoidance and row-end-turning modules, yielding success rates of 77.5% and 70% respectively. The final deliverables include a low-cost (PKR 242,000) working rover, a labelled under-canopy wheat dataset from on-campus plots, and a trained edge-deployable detection model, together demonstrating that camera-based,

GPS-free autonomy is a practical step toward earlier and more targeted responses to the pests and diseases that account for an estimated 20–40% of global crop losses each year.

ACKNOWLEDGMENTS

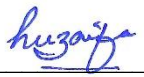
The authors wish to express their sincere appreciation to our advisor, Dr. Hassan Jaleel, and his research assistants Muhammad Ibrahim Rana and Muhammad Ahson Hassan for their continuous guidance, valuable feedback, and unwavering support throughout the course of this senior project and during the preparation of this manuscript.

We are also grateful to the Department of Electrical Engineering and the Syed Babar Ali School of Science and Engineering at LUMS for providing the laboratory facilities, equipment, and access to the on-campus wheat plots that made the data collection and field testing of RowSense possible. Special thanks are to Dr. Jahangir Ikram of the EE Department for his assistance with hardware fabrication and procurement, and to the farm staff for their help in maintaining the test environment during repeated field trials.

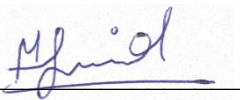
ORIGINALITY CERTIFICATE

“We the undersigned certify that this submission is the original work of members of the group and meets the Faculty's Expectations of Originality”:

1. Huzaifa Saeed [2020-10-0070]

Signature:  26100070@lums.edu.pk

2. Mohammad Junaid [2020-10-0391]

Signature:  26100391@lums.edu.pk

3. Mohid Joya [2020-10-0141]

Signature:  26100141@lums.edu.pk

4. Virad Munir [2020-10-0285]

Signature:  26100285@lums.edu.pk

Dr. Hassan Jaleel

hassan.jaleel@lums.edu.pk

Table of Contents

PROJECT ABSTRACT	II
ACKNOWLEDGMENTS	IV
ORIGINALITY CERTIFICATE.....	V
LIST OF FIGURES	VIII
LIST OF TABLES.....	IX
Chapter 1	1
Introduction	1
1.1: Motivation	1
1.2: Problem Statement.....	2
1.3: General Block Diagram	3
1.4: Social Benefits And Relevance	3
1.5: Goals	4
1.6: Objectives	4
1.7: Outcomes	5
1.8: Deliverables (Outputs)	6
1.9: Timeline And Distribution of Work.....	6
Chapter 2	8
Background	8
2.1: Literature Review.....	8
2.2: Market Survey (If applicable).....	12
Chapter 3	14
Methodology and Tools.....	14
3.1: System Level Design.....	14
3.2: Final Design.....	16
3.3: Tools/Instruments.....	17

Chapter 4	19
Project Simulation	19
4.1: Setup	20
4.2: Results.....	21
4.3: Result Analysis And Outlook.....	24
 Chapter 5	 26
Project Implementation	26
5.1: Hardware Setup.....	26
5.2: Experimental Procedure.....	29
5.3: Implementation Results.....	38
5.4: Result Analysis And Outlook.....	41
 Chapter 6	 43
Cost Analysis	43
 Chapter 7	 45
Conclusion and Future Recommendations.....	45
References.....	48

LIST OF FIGURES

FIG. 1 GENERAL BLOCK DIAGRAM.....	3
FIG. 2 GANTT CHART FOR FALL 2025	6
FIG. 3 GANTT CHART FOR SPRING 2026.....	6
FIG. 4 SYSTEM LEVEL DESIGN	13
FIG. 5 ROWSENSE ROVER FINAL DESIGN	15
FIG. 6 ROWSENSE ROVER PROTOTYPE	17
FIG. 7 ROVER TRAVERSING BETWEEN SPINACH ROWS	18
FIG. 8 SAMPLE OBJECT DETECTION USING YOLOV11 ON SPINACH.....	19
FIG. 9 NORMALIZED CONFUSION MATRIX (YOLOV11)	20
FIG. 10 PRECISION – RECALL CURVE (YOLOV11).....	20
FIG. 11 F1 – CONFIDENCE CURVE (YOLOV11).....	21
FIG. 12 CLASSIFICATION LOSS (YOLOV11).....	21
FIG. 13 BOX LOSS (YOLOV11).....	22
FIG. 14 UPDATED HARDWARE DESIGN – PROTOTYPE TO FINAL	25
FIG. 15 LABELLED VIEW OF ON-BOARD COMPONENTS	25
FIG. 16 WHEAT BEDS AND CROP ROWS (LUMS TEST PLOT)	27

FIG. 17 IMU BASED LINEAR TRAVERSAL PIPELINE	28
FIG. 18 OBSTACLE AVOIDANCE PIPELINE AND STATE MACHINE	29
FIG. 19 ROW END TURNING PIPELINE AND STATE SEQUENCE	30
FIG. 20 AUTONOMOUS OPERATION OVERVIEW	31
FIG. 21 WHEAT CROP CYCLE ACROSS THE SEASON	32
FIG. 22 ANNOTATED WHEAT IMAGERY (WHEAT HEAD, TILLER, LEAF STRIPE RUST)	33
FIG. 23 TILLER AND WHEAT HEAD DETECTION (YOLOV26).....	35
FIG. 24 PRECISION-RECALL AND F1-CONFIDENCE CURVES (YOLOV26)....	36
FIG. 25 YOLOV26 BOX AND CLASSIFICATION LOSS CURVES.....	36

LIST OF TABLES

TABLE 1 DISTRIBUTION OF WORK	7
TABLE 2 APPLIED AUGMENTATIONS (SPINACH, YOLOV11)	18
TABLE 3 WHEAT DATA COLLECTION PARAMETERS.....	32
TABLE 4 WHEAT DATA APPLIED AUGMENTATIONS	33
TABLE 5 CLASS INSTANCES BEFORE AND AFTER AUGMENTATION	33
TABLE 6 YOLOV26 TRAINING PARAMETERS	34
TABLE 7 OBSTACLE AVOIDANCE FIELD TRIAL STATISTICS.....	34
TABLE 8 ROW END TURNING FIELD TRIAL STATISTICS	35
TABLE 9 YOLOV26 CLASS PREDICTION RESULTS	36
TABLE 10 COST BREAKDOWN OF ROWSENSE ROVER	38

Chapter 1

INTRODUCTION

1.1: MOTIVATION

Modern agriculture faces increasing pressure to produce higher yields using fewer resources, including reduced water usage, minimal chemical inputs, and a shrinking labor force, all while maintaining crop quality and environmental sustainability. Pests and plant diseases remain one of the most significant threats to food security, accounting for an estimated 20–40% of global crop losses annually [1]. Early-stage symptoms—such as small lesions, discoloration, or subtle canopy deformation—often emerge at the plant or leaf level. Detecting these indicators at an early stage is critical, as timely intervention can prevent widespread infestation, reduce chemical usage, and substantially lower economic losses for farmers.

Aerial platforms such as UAVs have become valuable tools for large-scale farm monitoring, enabling rapid assessment of vegetation indices, thermal stress, and weed distribution over wide areas. However, UAV-based sensing is inherently limited by its top-down perspective. Early disease symptoms frequently lie beneath the upper canopy, particularly in dense row crops. Moreover, rotor downwash causes leaf motion thus blurring fine-grained visual features, especially those that are required for early detection of pests and disease. Due to these limitations, UAVs are less effective for identifying localized, early-stage issues. Our approach does not intend to replace UAVs, rather it complements aerial sensing by introducing an under-canopy, lateral viewpoint that directly observes plant stems, leaves, and soil-level features within crop rows.

Compact under-canopy ground robots are a practical solution that bridge this sensing gap. By traversing between crop beds, such platforms are able to collect high-resolution, plant-level imagery that is otherwise inaccessible to aerial systems. Furthermore, these robots can operate in environments where GPS accuracy is unreliable, such as under dense foliage or near tall trees, greenhouses, and farm infrastructure, thus motivating the use of vision-only navigation strategies

that rely on local visual cues, such as crop row structure and free-space appearance, rather than precise global positioning or heavy sensor suites. A lightweight, camera-based rover can therefore remain low-cost, energy-efficient, and mechanically simple, making it suitable for deployment in real-world farms.

At Lahore University of Management Sciences (LUMS), we implement this vision-centric concept on campus plots of vegetables and field crops using RowSense, a compact under-canopy rover designed to autonomously drive between crop rows. RowSense navigates using on-board camera and IMU feedback alone, without LiDAR, GPS, or SLAM, and captures near-canopy imagery from within the crop bed. Each traversal produces fine-grained visual data that is processed to generate within-row alerts, identifying early signs of pests, disease, or plant stress. In parallel, we developed a labelled under-canopy dataset tailored to local crop conditions, which is used to train and evaluate machine-learning models for plant health monitoring. This dataset and system together enable timely, targeted intervention, supporting preventive agriculture practices while reducing reliance on blanket chemical treatments.

Overall, this work addresses a critical gap between aerial monitoring and manual scouting by demonstrating that low-cost, vision-only autonomous rovers can provide actionable, plant-level intelligence under real farm conditions. By enabling early detection directly within crop rows and operating independently of GPS or complex sensor fusion, RowSense contributes toward scalable, data-driven precision agriculture suitable for resource-constrained environments.

1.2: PROBLEM STATEMENT

Design and implement a compact, low-cost, AI-enabled Unmanned Ground Vehicle that uses an on-board machine-learning model and autonomously traverses between crop beds to capture an under-canopy view. The rover should enable earlier, within-row detection of pests, diseases, and nutrient stress, support targeted data-guided inputs, and pave the way to safeguard yields in farms across Pakistan, while addressing the lack of affordable, locally developed under-canopy scouting solutions.

1.3: GENERAL BLOCK DIAGRAM

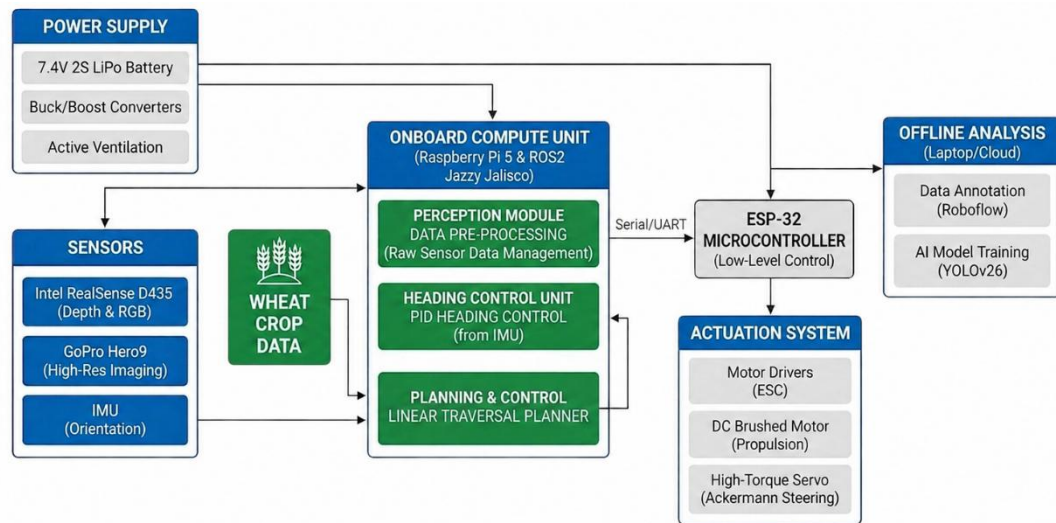


Fig. 1: General Block Diagram

The high-level architecture of RowSense is shown in Fig. 1. Power from a 7.4 V 2S LiPo battery is regulated through buck and boost converters to drive the on-board electronics, the steering servo, and the cooling fans, while the ESC receives the 7.4 V rail directly for the brushed DC drive motor. The ESP32-S3 microcontroller generates PWM signals for throttle and steering and can also receive Bluetooth Low Energy (BLE) commands from a companion mobile app for manual operation. Autonomous behaviour is computed on a Raspberry Pi 5 single-board computer, which streams synchronized depth and RGB frames from the Intel RealSense D435 stereo camera and quaternion data from a WheelTec N100 IMU. The Pi 5 fuses these into two parallel perception modules — row-end turning from the RGB stream and obstacle avoidance from the depth stream — and forwards the corresponding velocity commands to the ESP32-S3 over UART. In parallel, near-canopy GoPro Hero 9 imagery is annotated and augmented in Roboflow and fed to a YOLOv26 object-detection model whose output flags wheat heads, tillers, and stripe rust during a pass.

1.4: SOCIAL BENEFITS AND RELEVANCE

Our project, focused on under-canopy crop scouting, offers clear benefits to society. By providing earlier, within-row detection and targeted inputs, RowSense can help protect yields

while reducing unnecessary chemical, water, and energy use. The approach is practical for cases where UAVs overlook under-canopy details due to occlusion, hence a compact budget-friendly rover streamlines routine monitoring and detects hotspots that would otherwise be missed until visible damage has already occurred. In the Pakistani context, where wheat aphid infestations can cause yield losses of 20–80% and farmers often respond with blanket pesticide spraying, an under-canopy scouting tool that flags problem areas early can directly reduce chemical waste and protect both human health and the surrounding environment.

Deployment of an autonomous rover in fields demands compliance with certain safety measures such as supervised runs, safe speed limit and an accessible emergency stop. It also presents some legal considerations, such as the liability in the event of crop damage. Therefore, responsible use and upholding ethics in crop health assessment are necessary, requiring transparent communication of model limits and human verification before taking an action.

As regards to Sustainable Development Goals (SDGs), our project aligns with SDG 2 (Zero Hunger) by helping safeguard crop yields through earlier and resilient data-driven detection. It also promotes SDG 12 (Responsible Consumption and Production) by reducing excessive blanket spraying and providing targeted, data-guided inputs. These goals underscore the broader societal relevance of RowSense and its potential to contribute towards more sustainable, evidence-based crop management.

1.5: GOALS

Our aim is to make crop scouting faster, earlier, and more useful for real decisions in the field, without expensive infrastructure or complex workflows so that the yields are protected and unnecessary resource use is reduced.

1.6: OBJECTIVES

We have structured our project into clear, time-bound checkpoints to ensure we deliver a budget-friendly, compact under-canopy rover with actionable insights/field-ready recommendations. RowSense has the following objectives:

- To adapt and retrofit a sturdy base platform (chassis, motors, tyres) suitable for rough, uneven crop beds with custom sensing units (cameras, IMU), on-board computer, and sufficient battery storage.
- To collect and label raw under-canopy dataset using camera viewpoints with annotations for wheat heads, tillers, and visible disease symptoms such as stripe rust.
- To train and evaluate an on-edge ML model on the collected dataset so it reliably detects crop features and disease symptoms across varied growth stages and lighting conditions.
- To implement and validate the autonomous traversal of the rover between crop beds using IMU-based heading hold for straight-line motion, depth-based obstacle avoidance, and vision-based row-end turning.

1.7: OUTCOMES

Routine use of RowSense will enable field teams to monitor crop health simultaneously with its seasonal growth, detecting preliminary pest attacks and disease symptoms, and take timely and targeted measures to ensure a vigorous and thriving crop yield. The outcomes also include a clear demonstration that vision-only autonomous traversal — without LiDAR, GPS, or full SLAM — is feasible on a small four-wheel-drive platform under real outdoor conditions, providing a reusable open template for follow-on student projects, low-cost commercial systems for small farms, and further research on cooperative or swarm operation.

1.8: DELIVERABLES (OUTPUTS)

- An outsourced chassis, capable of traversing rough terrains, modified with a sturdy platform and 3D-printed cover to accommodate cameras, IMU, on-board computer and battery.
- A labelled under-canopy wheat dataset (1163 base images, 8265 after augmentation) collected from on-campus plots across an 11-week crop cycle, with annotations for wheat heads, tillers, and stripe rust.
- A locally trained YOLOv26 object-detection model, tested across the growth season and reported with performance metrics: per-class accuracy, precision, and recall.
- An autonomous rover with a vision-based and IMU-based navigation stack, validated through 40 field trials per behaviour on the LUMS wheat plots.

1.9: TIMELINE AND DISTRIBUTION OF WORK

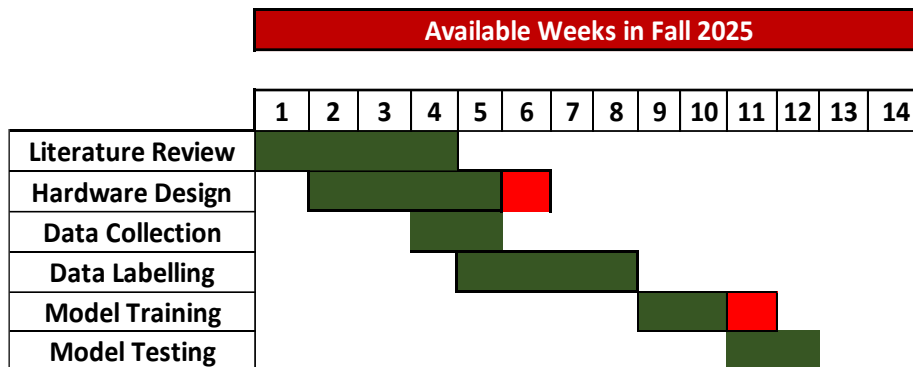


Fig. 2: Gantt Chart for Fall 2025

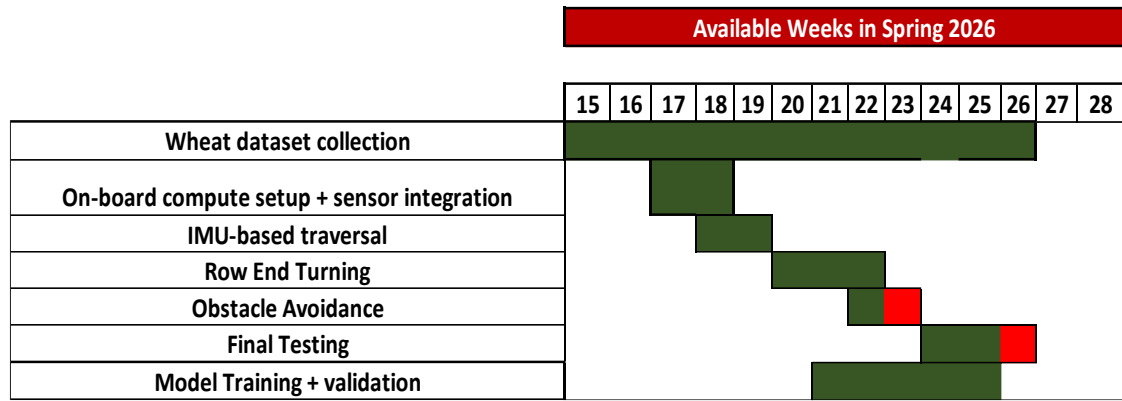


Fig. 3: Gantt Chart for Spring 2026

Table 1: Distribution of Work

Task	Responsible Person	Planned Duration
Literature Review	All members	Two Weeks
Hardware setup and sensor integration	Huzaifa Saeed Mohid Joya	Four weeks
Dataset acquisition	All members	Eleven Weeks
Annotation and Augmentation	Virad Munir Huzaifa Saeed Mohid Joya	Four Weeks
Model Training and Testing	Mohammad Junaid Virad Munir	Four Weeks
IMU-based Linear Traversal	Virad Munir Huzaifa Saeed	Two Weeks
Obstacle Avoidance	Virad Munir Mohammad Junaid	Three Weeks
Row End Turning	Virad Munir Mohid Joya	Two Weeks
Field Testing and Validation	All members	Two Weeks
Final Documentation	All members	Two Weeks

Chapter 2

BACKGROUND

2.1: LITERATURE REVIEW

Unmanned Ground Vehicles (UGVs) are increasingly used in agriculture to autonomously traverse crop rows and perform monitoring tasks that would otherwise be labour-intensive. Unlike drones (UAVs), ground rovers can operate under the canopy of tall crops such as maize, providing close-up inspection from soil level to the canopy top. Small under-canopy UGVs have been developed in the past few years for applications like high-throughput phenotyping, crop scouting, and even in-row operations like seeding cover crops. A prime example is the EarthSense TerraSentia rover, a compact four-wheeled robot designed at the University of Illinois for under-canopy data collection [2]. TerraSentia carries multiple sensors (RGB cameras and LiDAR) to capture plant traits such as height, stem width, and leaf area in real time. It can autonomously navigate between crop rows to scan about 10 plants per second throughout the day, enabling high-resolution 3D field phenotyping and yield prediction. This rover exemplifies the trend toward UGVs that gather detailed crop health data beneath dense crop foliage, which larger machinery or aerial drones cannot easily access.

Recent research prototypes mirror TerraSentia's goals. For instance, the Purdue AgBot (P-AgBot) is a small under-canopy robot developed to monitor corn and sorghum and even collect physical samples (e.g. leaf tissue) from within densely planted fields [3]. The P-AgBot navigates narrow corn rows where GPS signals are lost under the canopy; it overcomes this by using on-board LiDAR sensors for localization and row-following. A custom SLAM framework called P-AgSLAM processes dual 3D LiDAR data to map the cornfield's structure and estimate the robot's pose as it drives under the canopy. Whenever the robot emerges to areas with reliable GPS (e.g. field edges), the system integrates GPS updates to correct any drift in its localization. This approach of fusing local sensor-based navigation with occasional GPS illustrates how modern UGVs maintain accuracy in GPS-denied under-canopy environments. This example demonstrates that under-canopy UGVs have matured from concept to working prototypes in the

last 3–5 years, addressing the challenges of confined space navigation and continuous crop monitoring. Researchers emphasize that such robots, dedicated purely to monitoring, can operate continuously (even at night) and with minimal soil compaction due to their light weight, making them attractive for precision agriculture. However, a recent review noted that fully “turnkey” commercial UGV solutions for autonomous crop monitoring are still rare – most available platforms are either expensive, specialized for single tasks, or require custom integration. This gap in the market motivates low-cost research projects like RowSense, which aim to develop an affordable under-canopy rover that can detect crop issues (pests, weeds, nutrient deficiencies).

Advances in machine learning, particularly deep learning, have driven major improvements in crop pest and weed detection over the past five years. Convolutional Neural Networks (CNNs) are now a standard approach for plant image analysis, owing to their ability to automatically learn complex visual features. In weed management, CNN-based models dominate recent literature for identifying and segmenting weeds among crops. A 2023 survey of 61 studies (2019–2023) found that YOLO object detection models are the most widely adopted CNN architectures for weed detection, followed by networks like ResNet, Faster R-CNN, and MobileNet [4]. These models can process images from farm cameras or drones to distinguish crop plants from weeds, enabling precision herbicide application or robotic weeding. The challenge in weed detection is achieving high accuracy in varying field conditions and on lightweight hardware. Recent research addresses this by optimizing models for speed and robustness: one study demonstrated a real-time weed segmentation CNN deployed on an edge device (Nvidia Jetson), using model compression and knowledge distillation to maintain accuracy while running efficiently in the field. These efforts show that modern CNN models can feasibly run onboard small rovers like RowSense for on-the-spot weed identification.

In parallel, CNN-based computer vision has been applied to crop pest and disease detection. Pests (typically insects or larvae) are often small and camouflaged, making detection challenging. Recent approaches introduce attention mechanisms and multi-scale feature extraction to improve CNN performance on small pests. A study in 2022 used a GoogleNet CNN to detect pests from leaf images and integrated it into a smartphone app for farmers [5]. The app attained 93.8% classification accuracy, allowing Indonesian farmers to identify pest infestations via their phone cameras [5]. This illustrates how deep learning is not only used in research settings but is also

being transferred to accessible tools for growers. Beyond insects, CNN models have long been used to identify plant diseases (often manifesting as leaf spots or discoloration). The well-known PlantVillage dataset, for example, has enabled training of deep models to classify dozens of crop diseases from leaf photos with high accuracy [6]. Modern approaches extend this to detection *in field conditions*, dealing with varying lighting and backgrounds.

Detecting nutrient deficiencies and stress in crops is another area leveraging machine learning. Nutrient stress (e.g. nitrogen or potassium deficiency) often exhibits subtle symptoms like leaf colour changes, which can be quantified through imaging. Traditional vegetation indices (like NDVI) are useful, but CNNs can learn more complex spectral-spatial patterns of stress. For instance, a statistical segmented regression with a Mask R-CNN deep learning model detected early nutrient stress symptoms in cucumber seedlings [7]. This Mask R-CNN approach achieved an F1 score of ~93.4% in segmenting and identifying nutrient-deficient seedlings a few days after stress initiation [7]. This is significant because early detection allows corrective action (fertilization) before yield is impacted. Other studies report similar success: a Mask R-CNN could recognize nutrient deficiency symptoms in chili plants with about 82.6% accuracy [8], and a modified Faster R-CNN detected hydroponic lettuce nutrient stress with ~86% mean average precision [9]. These models typically analyse RGB images for symptoms, but some research leverages hyperspectral imagery for even earlier stress detection. In summary, modern machine learning techniques (especially deep CNNs) have become integral to crop monitoring: they enable automated identification of weeds for precision weeding, detection of pests and diseases for targeted pest management, and recognition of nutrient stress for timely agronomic interventions. The RowSense rover will incorporate such models (e.g. a CNN-based classifier or segmenter) on its onboard computer to detect pests or weeds in real time as it navigates between crop beds. By deploying trained models on the rover's cameras, the system can flag problem areas (like pest-infested plants or weed patches) and potentially alert farmers. Literature in the past five years gives confidence that lightweight deep learning models can achieve the necessary accuracy and speed for these tasks in the field.

Autonomous navigation in farms often relies on GPS (often RTK-GPS) for localization, but under dense crop canopies or in greenhouses, satellite signals can be occluded or degraded. Thus, SLAM and sensor fusion techniques have been crucial to enable UGV navigation without GPS.

A common approach in the last few years is to combine LiDAR and IMU sensors to perform SLAM in crop rows or orchards. LiDAR provides high-precision distance measurements that are invariant to lighting conditions. In agricultural settings, 2D LiDAR has been widely used for row detection and obstacle avoidance, for example by scanning crop rows to keep the robot centered and detect any obstacles like rocks or plants. However, 2D LiDAR alone cannot capture terrain elevation changes, which is problematic on uneven farm terrain. Newer systems use multi-plane or 3D LiDAR to map the full 3D structure of the environment (e.g. tree orchards or tall crops) for more robust localization. The trade-off is higher cost and computation for 3D LiDAR data. To bridge the gap, sensor fusion of LiDAR with an IMU (Inertial Measurement Unit) has gained traction to improve localization without an exorbitant cost or data burden. The IMU provides inertial angles and acceleration data that can be used to correct the LiDAR's point cloud for robot tilt and roll on bumpy ground. In fact, real-time tilt correction using IMU feedback can de-wobble LiDAR scans on uneven fields, yielding a more accurate and level map.

Unlike many agricultural robots that rely on LiDAR-IMU fusion or SLAM-based localization, RowSense adopts a purely vision-driven navigation strategy without performing explicit SLAM. Instead of building a global map or estimating full robot pose, RowSense focuses on reactive and local navigation policies derived directly from camera imagery. While camera-only navigation in outdoor farms is challenging due to illumination changes, shadows, wind-induced crop motion, and repetitive visual patterns, recent advances in learning-based perception and control enable reliable navigation without explicit mapping. Several studies have shown that end-to-end or modular vision-based approaches can follow crop rows by exploiting visual cues such as row symmetry, vanishing points, texture gradients, and free-space appearance, even under canopy and GPS-denied conditions. For instance, learned row-following methods such as CropFollow demonstrate that deep neural networks can directly predict steering commands from monocular images, eliminating the need for LiDAR or SLAM [10].

RowSense employs a single Intel RealSense D435 stereo depth camera and a WheelTec N100 IMU to continuously estimate relative alignment with the crop bed and detect obstacles using image-based techniques rather than metric localization. A path-frame transform anchors a corridor mask to the rover's intended trajectory so that obstacle detection remains robust even while the rover is in the middle of a heading correction. Row-end events are recognised by

detecting orange cone markers in the RGB stream via colour thresholding and contour analysis. This perception–action loop enables RowSense to remain centered between rows and avoid collisions by making incremental control decisions based solely on current visual and inertial input.

At a system level, RowSense follows a behaviour-based or reactive navigation paradigm rather than global path planning. High-level motion goals (e.g., “proceed down the row” or “turn at row end”) are executed using visual triggers such as cone detection, and inertial triggers such as accumulated yaw error and dead-reckoned lateral offset. This approach avoids drift accumulation and mapping failures commonly associated with SLAM in visually repetitive agricultural environments. Although vision-only, non-SLAM navigation may not provide globally consistent position estimates, it is well suited for structured row-crop fields where motion is largely constrained and decisions are local. The literature indicates that such approaches can achieve reliable row traversal with sub-metre lateral error using only onboard cameras, making them attractive for low-cost, power-efficient agricultural robots.

In summary, RowSense prioritises simplicity and robustness by avoiding LiDAR, GPS, and SLAM altogether. By relying exclusively on computer vision, a depth camera, and a single MEMS IMU, the rover can autonomously navigate crop rows, detect obstacles, and perform field monitoring tasks using minimal sensing hardware. This design choice aligns with RowSense’s objective of scalable, low-cost precision agriculture, demonstrating that effective autonomy is possible even without explicit mapping or multi-sensor fusion.

2.2: MARKET SURVEY (IF APPLICABLE)

A formal commercial market survey is not central to this project, as RowSense is primarily a research and prototyping effort rather than a market-ready product. However, existing commercial under-canopy systems (e.g., TerraSentia) remain costly and are not readily accessible to small farms or academic institutions in Pakistan. Imported turn-key under-canopy robots are typically priced well above one million PKR equivalent, and import duties, software licensing, and after-sales support raise the effective barrier even further. This lack of affordable,

locally supported solutions reinforces the relevance of developing a low-cost, modular rover that can be assembled, maintained, and iterated upon using commonly available components.

Chapter 3

METHODOLOGY AND TOOLS

3.1: SYSTEM LEVEL DESIGN

Fig. 4 summarizes the system level design of RowSense: a compact, AI-enabled UGV that autonomously scouts between crop beds under the canopy and flags potential lesions, infestations, and disease symptoms. The design rests on three complementary blocks: a UGV platform with a small footprint and low height; an artificial-intelligence block based on a YOLO object-detection family; and a sensing block built around cameras and a single IMU.

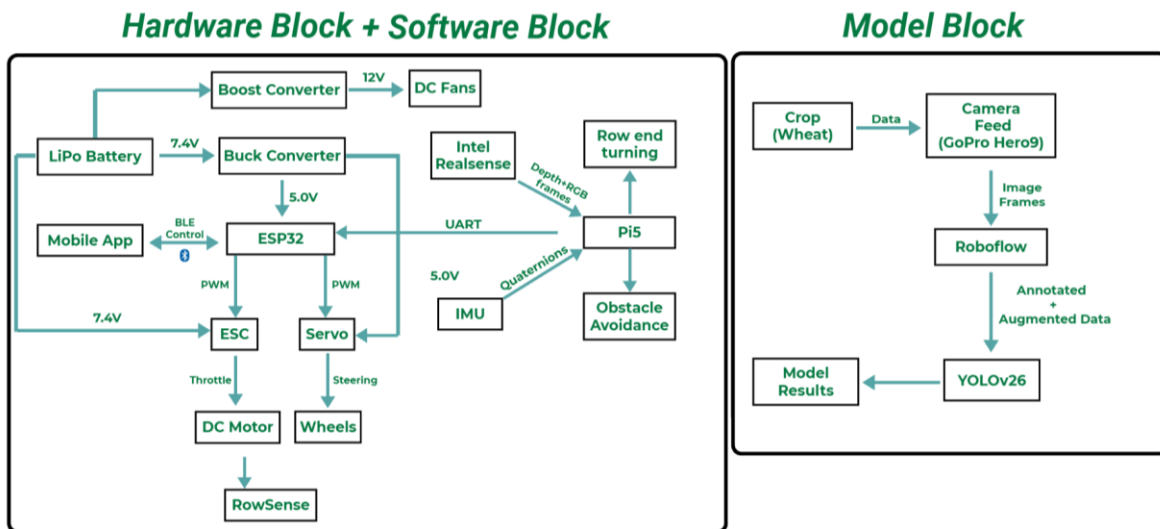


Fig. 4: System level design

3.1.1: Hardware Block

RowSense rover hardware centres on a four-wheel-drive chassis that carries all sensing, computing, and actuation hardware. A 7.4 V 2S LiPo battery (4000 mAh, 30C) supplies a direct 7.4 V rail to the Electronic Speed Controller (ESC), which drives the brushed DC traction motor. The same battery feeds two regulated rails through dedicated converters: a buck

converter that produces 5.0 V for the ESP32-S3 microcontroller and the steering servo, and a boost converter that produces 12 V for the cooling fans. A 10,000 mAh USB power bank powers the Raspberry Pi 5 single-board computer independently of the LiPo, isolating the high-current motor transients from the on-board compute.

Several design choices were weighed at this layer. A four-wheel-drive (4WD) configuration was preferred over a 2WD differential drive because the LUMS wheat plots contain raised beds with 40–50° embankments and loose, uneven soil; 2WD platforms could not maintain traction during in-row corrections. The original soft suspension and underpowered (2.8 kg-cm) steering servo were replaced with stiffer shocks and a 13–15 kg-cm high-torque servo to handle the payload and provide reliable steering authority under load. The black-box factory ESC was swapped for a configurable ESC suitable for 2S operation, allowing predictable current limits and brake behaviour. A custom acrylic base plate, followed by a 3D-printed cover were added on the chassis mounts to provide a rigid, vibration-damped surface for sensors.

3.1.2: Software (Control) Block

The software (control) block of the rover is split between two compute nodes. The ESP32-S3 acts as the real-time motion controller that generates PWM signals for the ESC (throttle) and the steering servo, and presents a Bluetooth Low Energy (BLE) interface to a companion mobile app for manual control in case of commissioning and emergency overrides. The Raspberry Pi 5 (8 GB RAM, 128 GB SD) runs the autonomy stack: it receives synchronized depth and RGB streams from the Intel RealSense D435 and quaternion orientation data from the WheelTec N100 IMU, computes velocity commands using three perception modules (heading hold, obstacle avoidance, row-end turning), and forwards them to the ESP32 over UART. Splitting hard real-time PWM generation onto a microcontroller while keeping perception on a Linux SBC simplifies both sides and matches the practical decoupling commonly found in robotics stacks.

A behaviour-based architecture was chosen over a full SLAM-based or end-to-end learned policy. Behaviours are independently testable, easy to interpret, and tolerant of failure (e.g., a missed cone simply degrades to heading hold, rather than corrupting a global map). The trade-off is that the rover cannot replan globally if a row is blocked end-to-end, but in practice this

is an acceptable limitation for an under-canopy scouting platform whose goal is straight-line traversal between known beds.

3.1.3: Model Block

RowSense model (perception) block is distributed into two parts. The first is the navigation perception that produces velocity commands in real time. The second is the agronomic perception: a YOLO-family object detector trained on a custom under-canopy wheat dataset to flag wheat heads, tillers, and stripe rust. We began with YOLOv11 during initial testing on a smaller spinach dataset before retraining on YOLOv26 with a much larger wheat dataset for the final run. YOLO was preferred over heavier two-stage detectors (Faster R-CNN, Mask R-CNN) because it is fast enough for near-real-time inference on the Pi 5 and is well supported on the Roboflow and Ultralytics ecosystem we used for annotation and training.

3.2: FINAL DESIGN

The final integrated rover combines all three blocks on a compact 4WD chassis with a green 3D-printed cover as shown in Fig. 5.



Fig. 5: RowSense Rover Final Design

3.3: TOOLS/INSTRUMENTS

3.3.1: Software Packages

- Roboflow Universe — used for image annotation, augmentation, and dataset management for both the spinach (YOLOv11) and wheat (YOLOv26) datasets.
- PyTorch and the Ultralytics YOLO toolkit — used for model training, validation, and inference; AdamW optimizer with learning rate 0.001 was used for YOLOv26.
- Arduino IDE — used to develop the ESP32-S3 firmware that bridges UART commands from the Pi 5 to PWM signals for the ESC and steering servo.
- Python on the Raspberry Pi 5 (with OpenCV, NumPy, pyrealsense2, and SciPy) — used to implement the IMU heading hold, depth-based obstacle avoidance, and HSV-threshold row-end turning pipelines.
- A custom BLE control mobile app — used for manual throttle / steering during commissioning and emergency takeover.

3.3.2: Hardware Instruments

- ESP32-S3 microcontroller — main motion controller; generates PWM and bridges UART.
- Brushed DC motor + configurable 2S-compatible ESC — propulsion and speed control.
- High-torque (13–15 kg-cm) servo motor — steering actuator.
- 7.4 V 2S 4000 mAh 30C LiPo battery — primary on-board power source.
- Buck converter (→ 5.0 V) and boost converter (→ 12 V) — regulated rails for compute and cooling.
- GoPro Hero 9 with adjustable height mount — high-frame-rate RGB capture used for dataset acquisition.
- Intel RealSense D435 stereo depth camera — on-board RGB-D perception for obstacle avoidance and row-end turning.

- WheelTec N100 IMU — quaternion orientation and yaw-rate feedback for heading hold and dead-reckoning.
- Raspberry Pi 5 (8 GB RAM, 128 GB SD card) + 10 000 mAh power bank — on-board compute and isolated power.
- Two 12 V DC fans — active cooling for the ESC, converters, and Pi 5.

3.3.3: Testing Equipment

- Bench power supply — powering and current-limiting the rover during indoor testing.
- Oscilloscope — inspecting PWM waveforms and UART traffic between the Pi 5 and the ESP32.
- Digital multimeter — verifying converter rails (5.0 V, 12 V) and continuity of the wiring harness.

Chapter 4

PROJECT SIMULATION

Since on-campus testing grounds were available throughout the project, no software-only simulation of the rover dynamics or sensor pipeline was carried out. Instead, the first project iteration acted as a real-world simulation environment for the full vision pipeline: a rover prototype was driven manually between rows of spinach on the LUMS plots, an initial under-canopy dataset was collected, and a first-generation YOLOv11 model was trained on this dataset to validate the feasibility of the detection pipeline before scaling up to the full wheat campaign. This chapter documents that initial simulation/iteration.



Fig. 6: RowSense Rover Prototype



Fig. 7: Rover traversing between spinach rows

4.1: SETUP

Spinach data was collected by GoPro Hero 9 Black at 2.7K resolution and 120 frames per second. The data was preprocessed for annotation with two labels defined: leaf and lesion. 123 images were annotated manually and were then augmented with the parameters in Table 2. After augmentation, the dataset expanded to 640 images. YOLOv11 was trained locally on this custom dataset with 100 epochs, 640 images and 2 batches.

Table 2: Applied Augmentations

Augmentations	Parameters
Flip	Horizontal
Crop	0% to 25%
Saturation	-32% to +32%
Brightness	-24% to + 24%
Blur	Up to 2.5px
Noise	1.96% pixels

4.2: RESULTS

A representative detection output is shown in Fig. 8, where the YOLOv11 model overlays “leaf” and “lesion” bounding boxes on a spinach frame. The aggregate quantitative behaviour of the model is summarised by the normalized confusion matrix in Fig. 9, the precision–recall curves in Fig. 10, and the F1–confidence curves in Fig. 11. Convergence behaviour is shown by the classification loss curves (Fig. 12) and the box loss curves (Fig. 13) over 100 epochs of training.

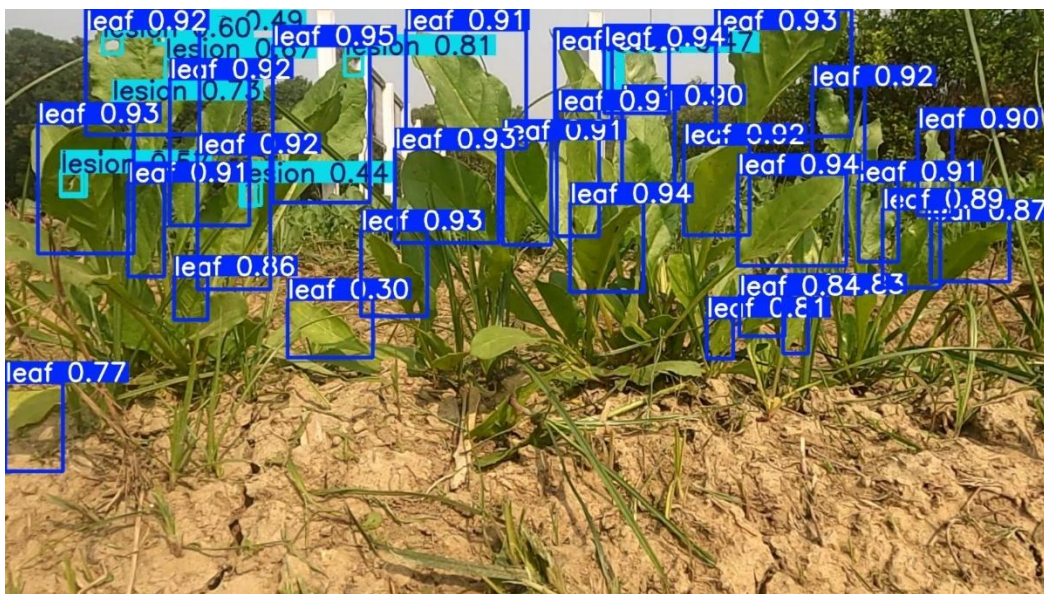


Fig. 8: Sample Object Detection using YOLOv11

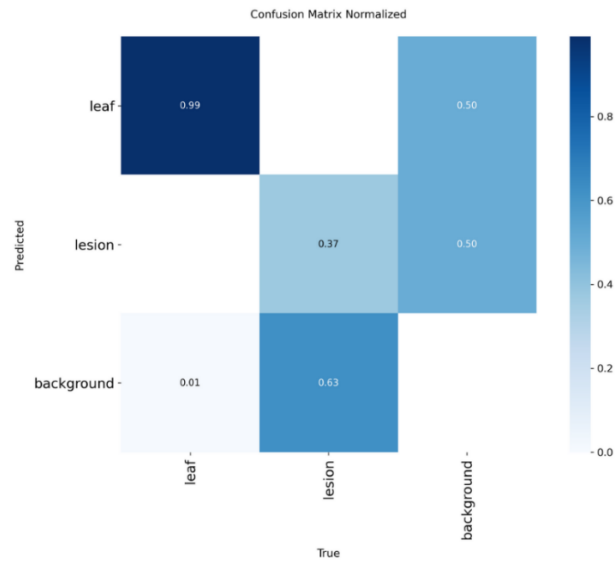


Fig. 9: Normalized Confusion Matrix (YOLOv11)

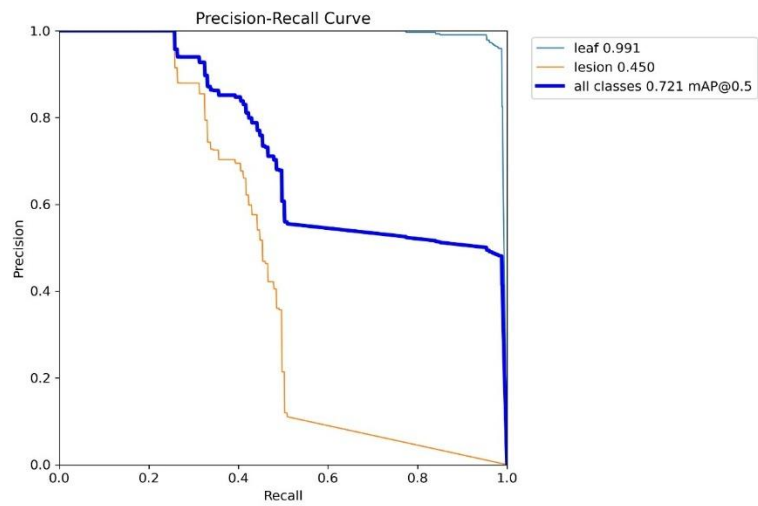


Fig. 10: Precision - Recall Curve (YOLOv11)

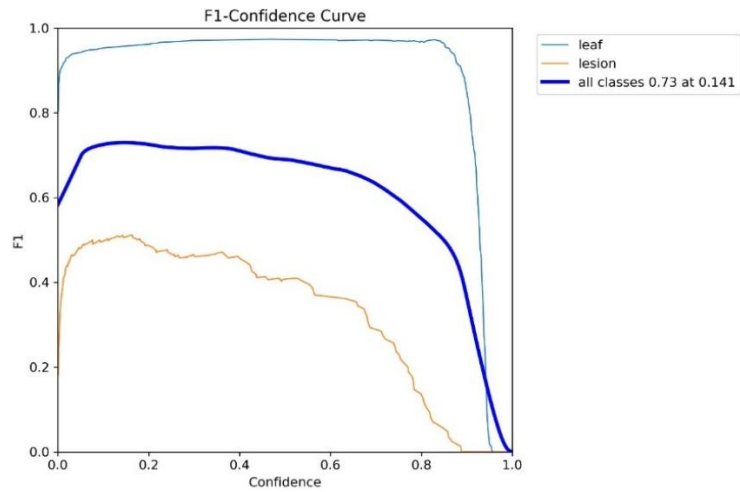


Fig. 11: F1 – Confidence Curve (YOLOv11)

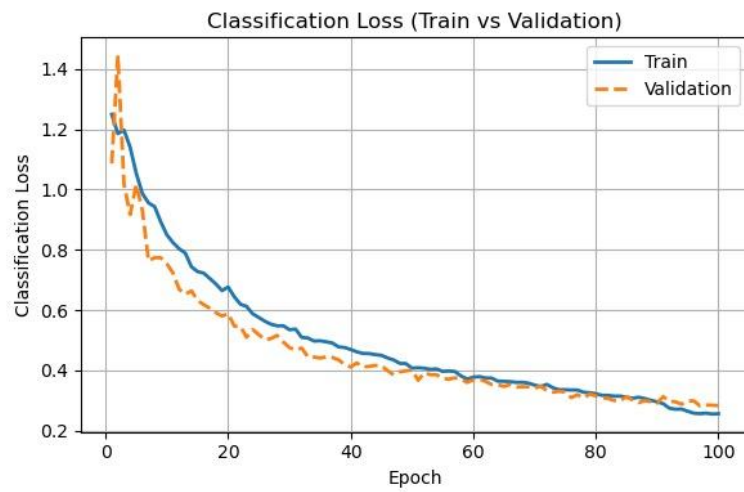


Fig. 12: Classification Loss (YOLOv11)

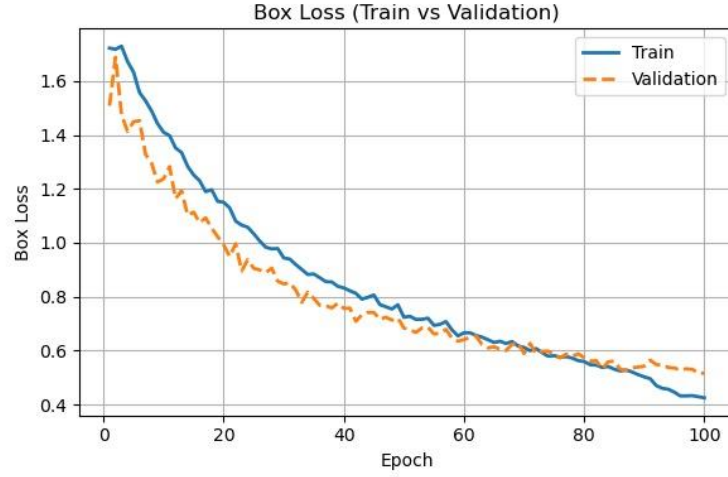


Fig. 13: Box Loss (YOLOv11)

4.3: RESULT ANALYSIS AND OUTLOOK

The model detects leaves with very high accuracy but misses out some lesions and has low accuracy lesion detection, as seen in Fig. 8. This is evident from the confusion matrix in Fig. 9, where leaves are almost perfectly classified, but most lesions are misclassified as background, revealing low lesion recall and class confusion. Leaf detection achieves almost perfect precision across all recall levels, as seen in the curves on Fig. 10, while lesion precision drops sharply as recall increases, reflecting the difficulty of detecting small lesions. Leaf F1 score in Fig. 11 stays consistently high across thresholds, while lesion F1 score peaks early and drops quickly, thus reflecting strong leaf detection but much weaker and confidence-sensitive lesion detection. Fig. 12 tells that the training classification loss declines steadily, showing that the model increasingly distinguishes between leaf and lesion classes and validation classification loss decreases consistently, confirming that class discrimination remains stable and does not overfit. Validation box loss drops in parallel with training box loss in Fig. 13, demonstrating good generalization of localization to unseen data.

So, the proposed system meets its core objectives of building a functional under-canopy sensing platform and training an edge-deployed vision model capable of reliable plant-level perception in real field conditions, and the strong and stable performance on leaf detection demonstrates the effectiveness of the collected dataset and validates the suitability of the YOLO

pipeline for routine rover runs. Simultaneously, the weaker performance on lesion detection highlights a key gap relative to the objective of early disease and nutrient-stress identification, hinting at the need for richer annotations and targeted data expansion. These initial tests support the feasibility of vision-based autonomous navigation and crop monitoring on a compact rover, and they clearly highlight perception of fine-grained symptoms as the primary area for further improvement before deployment of the decision-support dashboard.

Chapter 5

PROJECT IMPLEMENTATION

5.1: HARDWARE SETUP

RowSense uses a compact four-wheel drive rover platform designed to travel between crop rows and operate on uneven ground under the canopy. Its hardware was selected and modified to meet three core needs: reliable off-road mobility, sufficient steering and traction under load, and a stable mounting solution for sensing and control electronics.

5.1.1: Hardware requirements for under canopy traversal

RowSense required the following hardware features to achieve consistent motion in narrow row spacing and rough terrain:

- A compact size for between-row traversal
- A powerful motor with 4WD to maintain traction on loose soil and slopes
- A sturdy platform to carry the sensing payload and electronics without flexing
- A durable suspension system to absorb bumps and reduce vibration during motion

5.1.2: Identified shortcomings of the original chassis

Initial tests indicated the following four limitations which reduced performance and integration quality:

- Soft suspension, causing excessive bounce and reduced stability when carrying payload
- Underpowered steering servo, making turning unreliable on rough surfaces or when not moving
- Black box ESC with unconfigurable logic, limiting tuning and control behavior
- No base platform to mount sensing equipment securely, leading to unstable sensor placement

5.1.3: Modifications implemented on the chassis

To resolve the above issues, the rover was mechanically and electronically upgraded as follows:

- Stiffer shocks/springs were installed to improve weight handling and reduce oscillations over bumps
- The steering actuator was upgraded from a 2.8 kg-cm servo to a high torque 13 to 15 kg-cm servo, improving turning authority under load
- The original inaccessible ESC was replaced with a configurable ESC suitable for a 2S LiPo, enabling safer and more predictable drive control
- A 3D-printed cover was added on the existing mounting points to provide a compact surface that can fit the electronics and sensing hardware.

5.1.4: Updated hardware design and system integration

The progression from the prototype (red chassis, GoPro only) to the final build (green 3D-printed cover, Intel RealSense D435, IMU, Raspberry Pi 5) is shown in Fig. 14. A top-down view of the fully integrated electronics is shown in Fig. 15.

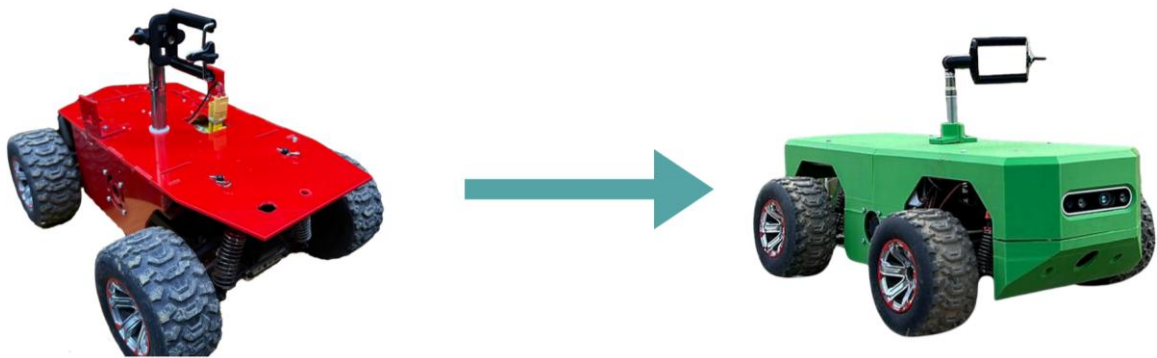


Fig. 14: Updated Hardware Design – Prototype to Final

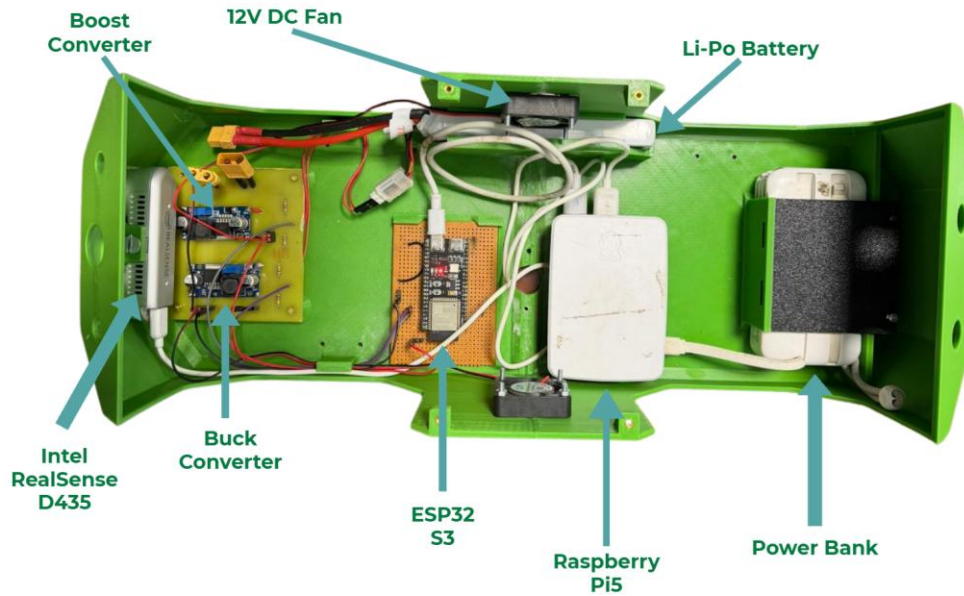


Fig. 15: Labelled View of On-Board Components

The final integrated setup consists of the following subsystems:

- **Control and processing:** an ESP32-S3 microcontroller acts as the real-time motion controller, generating PWM for throttle and steering and bridging UART traffic from the Raspberry Pi 5. The Raspberry Pi 5 (8 GB RAM, 128 GB SD) runs the autonomy stack and performs all perception in Python with OpenCV, pyrealsense2, and NumPy.
- **Mobility subsystem:** a brushed DC motor with a 2S-compatible configurable ESC drives the four wheels through the existing drivetrain; the high-torque servo provides steering. The ESP32-S3 outputs one PWM channel for the ESC and one for the servo.
- **Power subsystem:** a 7.4 V 2S 4000 mAh 30C LiPo is the primary source with the ESC drawing 7.4 V directly. A buck converter is used to produce 5.0 V for the ESP32-S3 and steering servo, while a boost converter is used to produce 12 V for the two DC cooling fans. A compact 10 000 mAh USB power bank supplies power to the Raspberry Pi 5 to isolate the on-board compute from motor current transients. All subsystems share a common ground.
- **Sensing subsystem:** a GoPro Hero 9 on an adjustable mount is used for dataset acquisition (high frame rate, 2.7K). An Intel RealSense D435 stereo depth camera provides aligned RGB and depth streams for obstacle avoidance and row-end turning. A WheelTec N100 IMU sends quaternion orientation and yaw-rate data over UART for heading hold and dead-reckoning.

- **Thermal management:** a pair of 12 V DC fans improve airflow around the heat-generating components (ESC, converters, and the enclosed Pi 5) and are powered via the regulated supply, so that cooling remains consistent throughout battery discharge.

5.2: EXPERIMENTAL PROCEDURE

Experiments were carried out on the LUMS wheat plots, consisting raised beds that are 0.6 m wide and 30.48 m long. The plots contain 10 crop rows separated by embankments of 0.23–0.28m height inclined at 40–50° (Fig. 16). These embankments, along with natural dynamic obstacles produce a highly unstructured terrain that introduces significant visual and sensor noise into the perception pipeline. Lighting also varies substantially between morning, midday, and late afternoon runs.



Wheat beds



Crop rows

Fig. 16: Wheat Beds and Crop Rows (LUMS Test Plot)

Autonomous operation — overview

RowSense autonomously traverses the crop rows based on three cooperating behaviours: an IMU-based heading-hold controller to maintain a straight line down the row, a depth-based obstacle-avoidance behaviour that drifts around obstacles inside the bed, and a vision-based row-end turning behaviour that uses orange cone markers to recognise the headland and perform a 90°–90° turn into the next row. Row-end turning takes precedence over heading hold once cones are confirmed, while obstacle avoidance temporarily overrides heading hold

whenever the corridor mask reports a blocked frame. A high-level diagram of the full autonomous-operation loop is given in Fig. 20.

5.2.1: IMU-based linear traversal

The IMU-based heading-hold pipeline (Fig. 17) reads quaternions from the WheelTec N100 IMU over UART at 100 Hz and converts them to a yaw angle. During an initial standstill calibration, 100 yaw samples are accumulated, and their circular mean is stored as the target heading for the row. The instantaneous heading error is computed as the wrapped difference between the latest yaw and this target. A discrete PID controller is then applied: the proportional term reacts to the current heading error ($K_p = 3.0$), the integral term cancels slow drift but is clamped to ± 0.50 to prevent windup ($K_i = 0.5$), and the derivative term uses the IMU's gyro rate directly ($K_d = 0.5$) rather than a noisy numerical derivative of the heading error.

Moreover, a cross-track term is added to pull the rover back to the centre of the bed even when its heading is already correct. A simple dead-reckoning update integrates the IMU-reported linear velocity along the bed and projects the wrapped heading error into a lateral offset ($\text{lateral_offset} += v \cdot \sin(ye) \cdot dt$), which is then compared against a fixed setpoint of 0 to compute the cross-track error. Both PID and cross-track outputs are summed and clamped to $\pm \text{MAX_ANGULAR_CMD} = 0.90$ rad/s before being published as `/cmd_vel.angular.z` to the ESP32. This ensures that small standing yaw errors do not add up to large lateral excursions during the 30 m run down a bed.

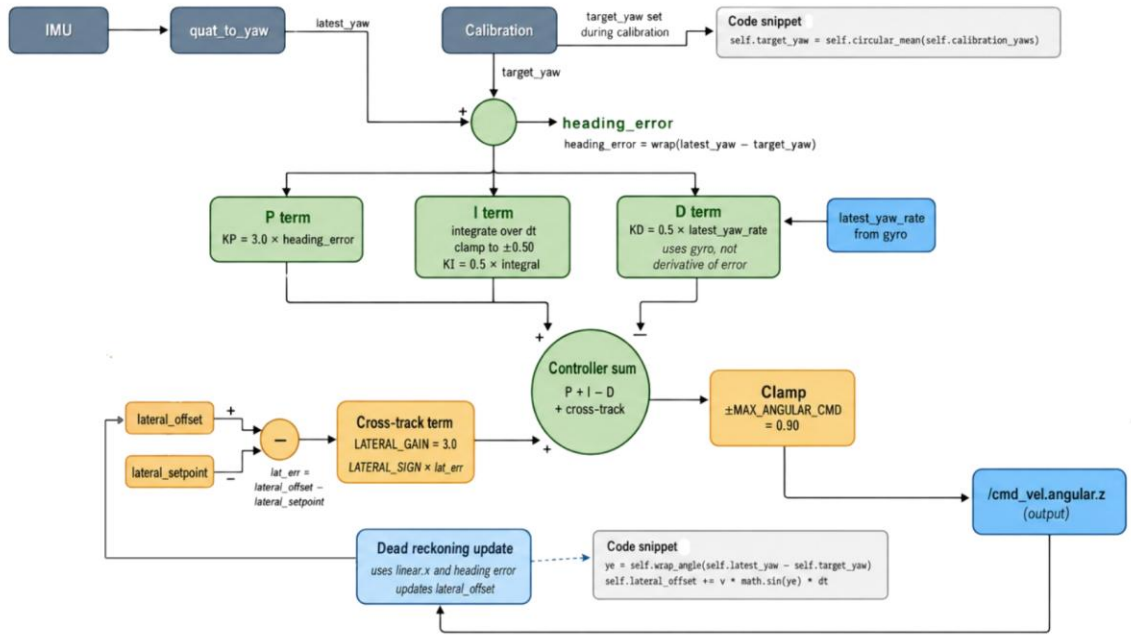


Fig. 17: IMU-based Linear Traversal Pipeline

5.2.2: Obstacle avoidance

The obstacle-avoidance pipeline operates on the RealSense D435 depth stream at 30 Hz. Each 16-bit depth frame is converted to metres, and the top 35% of pixels are clipped to remove the sky and far-field background. Remaining pixels are back-projected to 3D points in the camera frame using the intrinsic parameters:

$$Z = \text{depth}$$

$$X = \frac{(u - c_x)Z}{f_x}$$

$$Y = \frac{(v - c_y)Z}{f_y}$$

A 1.34° pitch correction is applied to level the camera, and a path-frame transform then rotates the point cloud by the current yaw error and shifts it by the dead-reckoned lateral offset so that the corridor mask follows the rover's intended trajectory rather than its instantaneous body frame — the central design choice that fixed the false-obstacle problem caused by flanking embankments seen in earlier iterations.

On the transformed cloud, three masks are applied: a corridor mask ($|X_{\text{path}}| < 14$ cm) for points lying within the bed, a left-shoulder mask ($-30 \text{ cm} < X_{\text{path}} < -14$ cm) and a right-shoulder mask ($+14 \text{ cm} < X_{\text{path}} < +30$ cm) for embankment-flanking detections. Common filters then enforce a minimum height-above-ground ($H_{\text{above}} > 5$ cm), a maximum forward distance ($Z_{\text{path}} < 4$ m), exclusion of a known embankment band, removal of low-height shoulder noise, and a 3×3 morphological open to remove single-pixel artefacts. The system finally requires ≥ 35 corridor pixels in the cluster and 3 consecutive confirming frames before declaring an obstacle, which dramatically reduces false positives from fallen leaves and dust on the lens.

Once an obstacle is confirmed, a five-state avoidance state machine is entered (Fig. 18). HEADING_HOLD is the default; on a confirmed obstacle the rover enters DRIFT_OUT, which uses a smoothstep ramp ($3x^2 - 2x^3$) to drift to the side shoulder over 1.0 s; TRAVERSE then holds at the shoulder until enough clear frames are observed; DRIFT_BACK ramps back to the centre with the same smoothstep profile; and HEADING_HOLD resumes. Whenever both shoulders are blocked simultaneously, or distance to the obstacle drops below 0.30 m, the state machine drops into STOPPED (zero velocity, wait for clear or operator intervention) regardless of the current state.

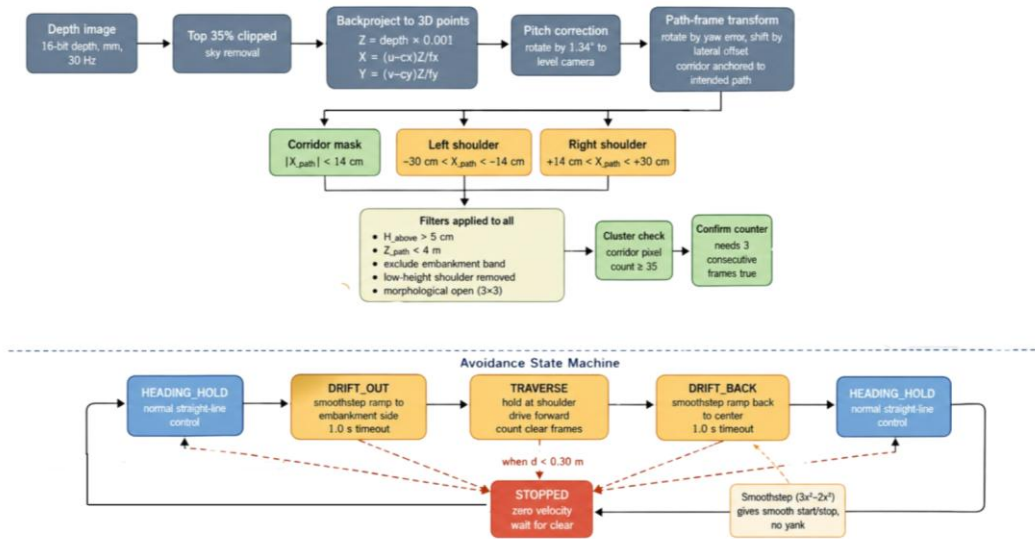


Fig. 18: Obstacle Avoidance Pipeline and State Machine

5.2.3: Row end turning

The row-end turning pipeline operates on the RealSense colour stream at 10 Hz and uses orange traffic cones as headland markers. RGB frames are converted from BGR to HSV (since hue is not affected by brightness), and an `inRange` threshold ($H \in [5, 22]$, $S \geq 120$, $V \geq 120$) extracts the orange cone pixels. A 3×3 morphological OPEN-then-CLOSE cleans the mask, after which `findContours` produces candidate cone blobs. A filter loop only keeps contours with area greater than or equal to 400 px and aspect ratio that is at least 0.7 so that cones are not confused with horizontal stripes of orange ground cover. The centroid of each surviving contour is computed via image moments, and the depth at the centroid is looked up from the aligned depth frame using a 5×5 patch median for robustness. A scoring function ($\text{area} / \max(0.2, \text{distance})$) selects the best candidate. The detection is finally confirmed by requiring it on 2 consecutive frames.

Once a cone is confirmed and within 3 m, the row-end state sequence in Fig. 19 is entered: `CONE_APPROACH_1` (drive toward the first cone with reduced heading-hold gains), `TURN_1` (pivot 90° toward the next row when distance ≤ 1.3 m), `TRANSIT` (drive across the headland for at most 18 s or until the second cone is seen), `CONE_APPROACH_2` (approach the second cone), `TURN_2` (pivot another 90° into the next row when distance ≤ 1.3 m), and finally `HEADING_HOLD` resumes. On entry to each turn, the `target_yaw` is shifted by $\pm 90^\circ$, with the sign alternating by a turn-pair counter so that the rover snakes through successive headlands. Several timeouts and recovery transitions are added — if a `TURN` times out without bringing the heading error below 5° , the controller falls through ($\text{err} \leq 30^\circ$) or escalates to `STOPPED` ($\text{err} > 30^\circ$).

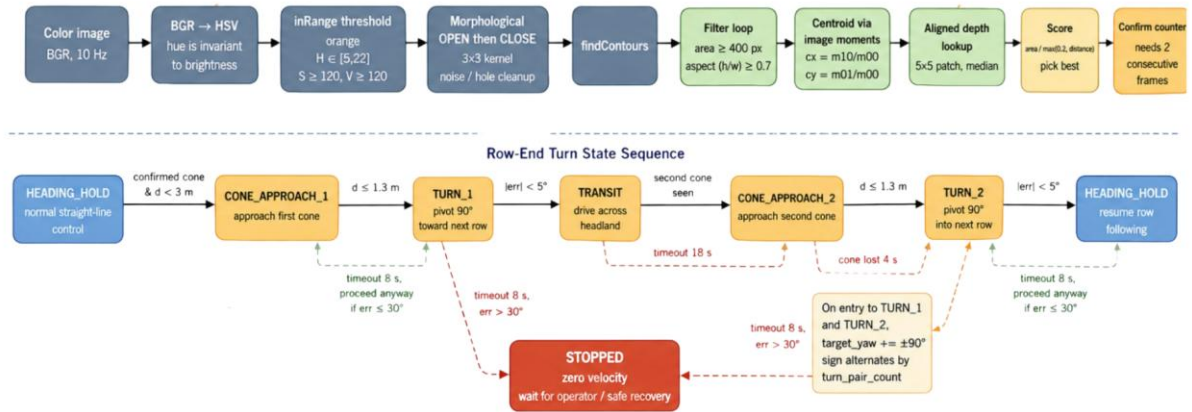


Fig. 19: Row End Turning Pipeline and State Sequence

5.2.4: Overall autonomous loop

Fig. 20 stitches all three behaviours together. After IMU calibration with 100 samples, the IMU heading hold maintains a straight line down the bed as the default behaviour, sharing the depth and RGB streams with the two reactive modules. Obstacle avoidance reads the depth stream and runs the five-state machine described above; row-end turning reads the RGB stream and runs the cone-approach to turn to transit sequence. Both modules write angular-velocity commands back to the heading-hold node, which arbitrates and forwards them to the ESP32-S3.

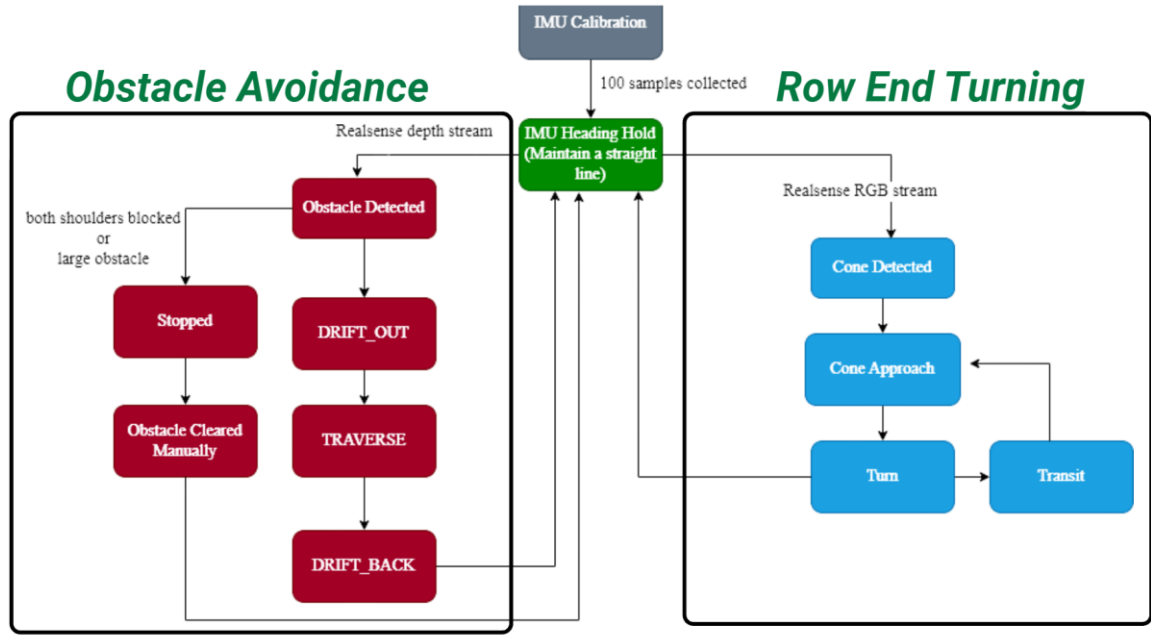


Fig. 20: Autonomous Operation Overview

5.2.5: Dataset acquisition for crop monitoring

In parallel with the autonomy work, a fresh under-canopy dataset was collected on the LUMS wheat plots over an 11-week period from January to April covering the Galaxy wheat variety from emergence through grain ripening. Data was captured by mounting the GoPro Hero 9 on the rover and driving it manually between two adjacent wheat rows; representative frames from week 1, week 5, and week 11 are shown in Fig. 21. A total of 11 datasets (one per week) were captured. The collection parameters are summarised in Table 3.



Fig. 21: Wheat Crop Cycle Across the Season

Table 3: Wheat Data Collection Parameters

Parameter	Value
Wheat Type	Galaxy Variety
Number of wheat rows used for dataset acquisition	2
Total number of datasets (weekly)	11
Dataset Collection Period	January – April
Target Features	Wheat heads, tillers, leaf stripe rust

A total of 1163 images were manually annotated in Roboflow Universe with three target classes — wheat heads, tillers, and wheat stripe rust — then expanded to 8265 images using the augmentation pipeline summarised below. Fig. 22 shows the annotation interface and Table 4 shows the augmentations applied; Table 5 summarises the class-instance counts before and after augmentation. Stripe rust is heavily under-represented in the raw dataset because it appears only in the later weeks of the cycle, which is why augmentation pushes its sample count by a factor of nearly $86 \times$ (vs. $2.3 \times$ for the wheat-head class).



Fig. 22: Annotated Wheat Imagery (Wheat Head, Tiller, Leaf Stripe Rust)

Table 4: Wheat Data Applied Augmentations

Augmentations	Parameters
Flip	Horizontal + Vertical
Crop	0% to 25% Zoom
Rotation	-15 to +15°
Brightness	-24% to + 24%
Blur	Up to 2.5px
Noise	Upto 1.92% pixels
Shear	15°

Table 5: Class Instances Before and After Augmentation

Label	Before Augmentation	After Augmentation
Wheat Head	13,051	29,846
Tiller	4,909	11,445
Wheat Stripe Rust	68	5,845

5.2.6: Model training

YOLOv26 was trained locally on the augmented wheat dataset with the parameters summarised in Table 6. The yolo26m variant was selected as a balance between accuracy and inference latency on the Pi 5. Training used the AdamW optimizer with a learning rate of 0.001 and binary cross-entropy loss, with a relatively high box-loss gain (7.5) compared to the classification-loss gain (1.5) since accurate localisation of small wheat heads matters more for downstream usage than fine class margins. Early stopping was governed by a patience parameter of 40 epochs over a 175-epoch upper bound.

Table 6: YOLOv26 Training Parameters

Parameter	Value
-----------	-------

Model	Yolo26m
Epochs	175
Image Size	640 px
Batch Size	8
Classification Loss Gain	1.5
Box Loss Gain	7.5
Optimizer	AdamW (learning rate = 0.001)
Loss Criterion	Binary Cross Entropy
Early stopping Patience	40 epochs

5.3: IMPLEMENTATION RESULTS

5.3.1: Field trial statistics

Two head-to-head field campaigns of 40 trials each were carried out on the LUMS wheat plots for the obstacle-avoidance and row-end-turning behaviours, with detection range set to 1.3–2.0m for both pipelines. The obstacle-avoidance behaviour was tested against two obstacle types (a static box and a low-profile bag of soil) and the row-end-turning behaviour against two orange cone markers placed at the headlands. The aggregated statistics are summarised in Table 7 and 8.

Table 7: Obstacle Avoidance Field Trial Statistics

Metric	Field Trial results
Total Trial Runs	40 Runs
Successful Identifications	31 Cases
Number of Obstacles	2
Detection Range	1.3m to 2.0m
Success Rate	77.5 %

Table 8: Row End Turning Field Trial Statistics

Metric	Field Trial results
Total Trial Runs	40 Runs
Successful Identifications	28 Cases
Number of Markers	2
Detection Range	1.3m to 2.0m
Success Rate	70 %

5.3.2: Model results

Representative qualitative outputs for the trained YOLOv26 model are shown in Fig. 23 — a tiller-detection frame on the left and a wheat-head detection frame on the right. Quantitative per-class results are summarised in Table 9, while Fig. 24 shows the aggregated precision–recall and F1–confidence curves, and Fig. 25 shows the box-loss and classification-loss curves over training.



Fig. 23: Tiller and Wheat Head Detection (YOLOv26)

Table 9: YOLOv26 Class Prediction Results

Class	Accuracy (%)	Precision (%)	Recall (%)
Wheat Heads	76	81.8	77.3
Tillers	74	60.8	70.2
Leaf Stripe Rust	97	97.3	97.3

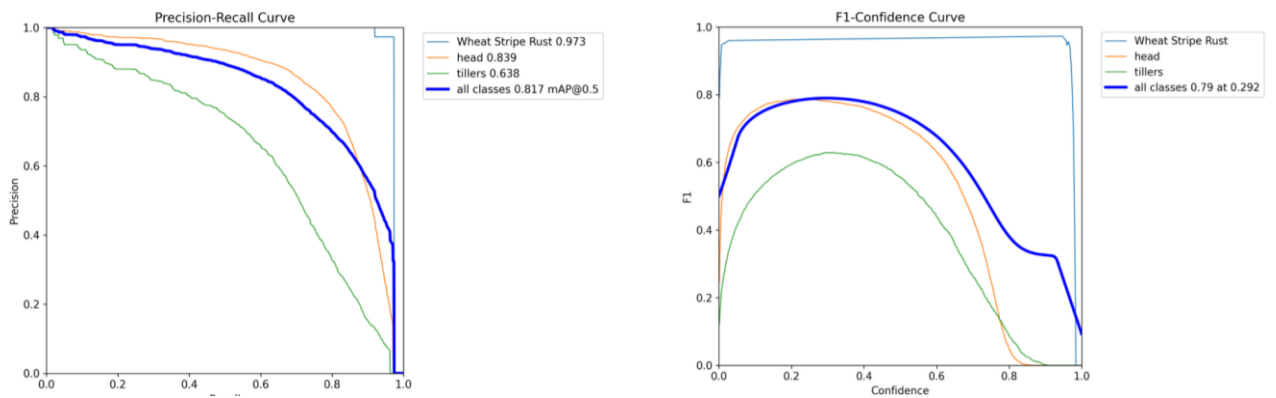


Fig. 24: Precision-Recall and F1-Confidence Curves (YOLOv26)

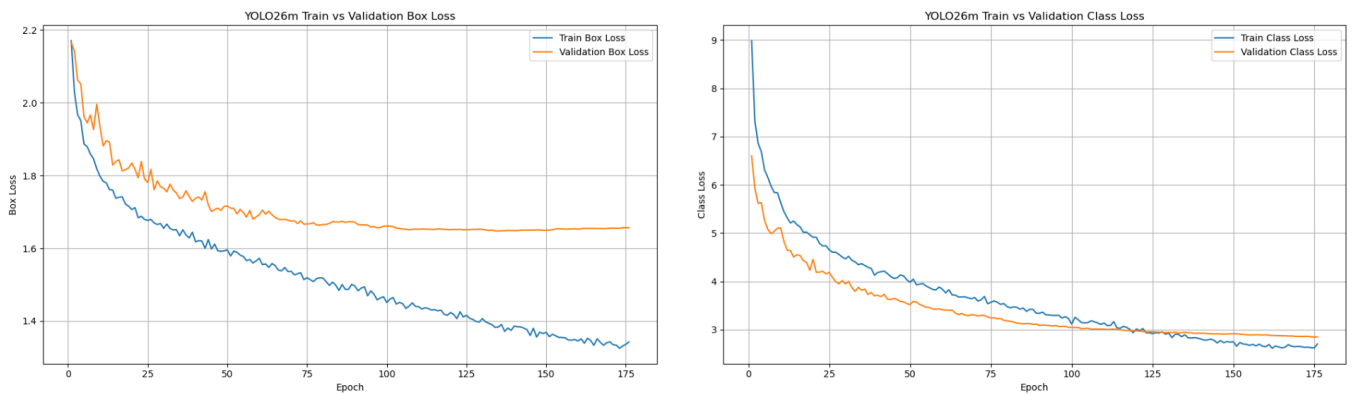


Fig. 25: YOLOv26 Box and Classification Loss Curves

5.4: RESULT ANALYSIS AND OUTLOOK

The field trial statistics in Table 7 and 8 demonstrate that the behaviour-based autonomy stack is reliable enough for routine scouting use. Obstacle avoidance succeeded in 31 of 40 trials (77.5%), with most failures attributable to either obstacles smaller than the $H_{\text{above}} > 5$ cm filter (which were under-counted into the corridor mask) or to the rover entering DRIFT_OUT too late after a confirmed detection. Row-end turning succeeded in 28 of 40 trials (70%), with most failures attributable to direct sunlight on the cones, which moves the orange hue outside the $H \in [5, 22]$ band, and to cones occluded by tall wheat heads near maturity. These two failure modes were not statistically independent — several afternoon runs failed both behaviours — indicating that lighting is the single most influential confound on overall mission success. Regardless of these constraints, the rover successfully maintained traction, balance, and controlled motion on the embankments throughout, with no chassis failures or sensor disconnections, thus confirming the mechanical reliability of the upgraded platform.

On the perception side, Table 9 shows that the YOLOv26 model achieves very strong performance on wheat stripe rust with 97% accuracy, precision, and recall (a near-perfect detector for the project’s core agronomic target), strong performance on wheat heads with 76% accuracy, 81.8% precision, 77.3% recall, and moderate performance on tillers with 74% accuracy, 60.8% precision, 70.2% recall. The precision–recall curves in Fig. 24 confirm this ordering, with mean average precision (at 0.5 threshold) of 0.817 across all classes and the tiller class trailing the others. The lower tiller score is due to two factors: tillers look like the surrounding green canopy of wheat leaves at the same stage, and the bounding boxes are inherently ambiguous i.e. a single tiller can be partially occluded by neighbouring leaves. Loss curves in Fig. 25 show that the training box loss decreases smoothly while the validation box loss plateaus at a higher value, showing mild overfitting on localisation but stable classification while the training and validation classification losses converge at similar values toward the end of training.

These results confirm that the proposed system meets its core objectives. The rover successfully performs autonomous-traversal with quantitative per-behaviour success rates of at least 70%. The perception objective is met by the YOLOv26 model trained on a

substantially larger and more class-balanced dataset than the spinach baseline, with stripe-rust detection in particular hitting precision and recall above 97%, directly addressing the most actionable agronomic class for early intervention.

Chapter 6

COST ANALYSIS

A detailed bill of materials for the final RowSense rover is provided in Table 7. All prices are given in Pakistani Rupees (PKR) and reflect the local market price at the time of procurement (late 2025 / early 2026). The total project cost was PKR 242,000, of which roughly 70% (PKR 170,000) is concentrated in three off-the-shelf imported components — the GoPro Hero 9 (PKR 85,000), the Intel RealSense D435 (PKR 60,000), and the Raspberry Pi 5 (PKR 25,000).

Table 10: Cost Breakdown of the RowSense Rover

Component	Cost (PKR)
Outsourced chassis (4WD platform with motors and suspension)	30,000
ESC + high-torque servo + ESP32-S3	10,000
7.4 V 2S 4000 mAh LiPo battery	12,000
GoPro Hero 9 (with adjustable mount)	85,000
Raspberry Pi 5 (8 GB RAM, 128 GB SSD)	25,000
Intel RealSense D435 depth camera	60,000
USB Power Bank (10,000 mAh)	5,000
3D-printed cover and acrylic base plate	7,000
WheelTec N100 IMU	8,000
TOTAL	242,000

For comparison, EarthSense’s TerraSentia [2], the closest commercial analog of RowSense, is priced well into the high five-figure US-dollar range when import duties, software licensing, and after-sales support are included, which translates to a price several times higher than RowSense for an institution in Pakistan. Research-grade prototypes such as Purdue’s P-AgBot [3] are not commercially available and rely on relatively expensive 3D LiDAR rigs (often tens

of thousands of US dollars on their own). RowSense, by contrast, achieves comparable under-canopy traversal capability (IMU-stabilised straight-line motion, reactive obstacle avoidance, and headland turning) with a depth camera, an IMU, and a hobby-grade chassis, while using a Raspberry Pi 5 as the on-board compute. The total bill of materials is well within the procurement budget of a Pakistani academic research lab, and the spec is competitive with commercial systems on the dimensions that matter most for under-canopy scouting: footprint, manoeuvrability on raised beds, and the ability to deploy a custom on-board ML model.

It is also worth noting that the GoPro Hero 9, while listed at PKR 85,000, is used only for high-frame-rate dataset acquisition during runs. In a production-oriented version of RowSense, the GoPro could be replaced with the existing RealSense D435's colour stream (or with a lower-cost rolling-shutter USB camera), bringing the total cost below PKR 150,000 with no meaningful change in detection performance on the targeted classes.

Chapter 7

CONCLUSION AND FUTURE RECOMMENDATIONS

This project set out to design and implement a compact, low-cost, AI-enabled Unmanned Ground Vehicle for under-canopy crop scouting on the LUMS wheat plots, addressing the practical gap between aerial UAV monitoring and manual scouting in Pakistani agriculture. Over two semesters of work, RowSense evolved from a manually controlled, BLE-driven 4WD platform tested on spinach with a YOLOv11 leaf/lesion baseline, into a fully autonomous, depth- and IMU-driven rover capable of traversing wheat beds, avoiding obstacles, and turning at row ends, with a custom YOLOv26 detector trained on a season-long Galaxy-variety wheat dataset.

Against the objectives stated in Chapter 1, the project achieves the following:

1. Adapt and retrofit a sturdy base platform with custom sensing, on-board compute, and battery storage - the chassis was upgraded with stiffer suspension, a high-torque steering servo, a configurable ESC, a custom acrylic base plate, and a 3D-printed cover, and all required electronics (RealSense D435, WheelTec N100 IMU, Raspberry Pi 5, ESP32-S3, 2S LiPo, and USB power bank) are integrated and field-tested.
2. Collect and label a raw under-canopy dataset - 1163 manually annotated images expanded to 8265 after augmentation across an 11-week crop cycle.
3. Train and evaluate an on-edge ML model - YOLOv26 reaches 97% accuracy on wheat stripe rust, 76% on wheat heads, and 74% on tillers, with stripe rust — the agronomically most actionable class — effectively solved.
4. Implement and validate autonomous traversal - heading-hold, obstacle avoidance (77.5% success rate over 40 trials), and row-end turning (70% success rate over 40 trials) all run on the Raspberry Pi 5 in real time.

The objectives that remain partially open are the originally planned nutrient-stress class (which we deferred in favour of the stripe-rust class once it became clear from the initial test results that rare-symptom classes need much larger collection windows than a single semester) and a fully integrated decision-support dashboard (which currently exists only as offline plotting of

detections rather than an in-field UI). These gaps are well bounded and form the natural backlog for follow-on work.

Future Recommendations

- Add localisation and mapping. The current behaviour-based stack has no global position estimate. A future iteration should add GPS at the headlands (where signal is available) and visual odometry inside the rows, allowing the rover to geotag detections so that hotspots can be revisited.
- Build a decision-support dashboard. Detections should be aggregated into a per-row, per-week dashboard with the geotagged location of each stripe-rust cluster, supporting targeted spraying instead of blanket application.
- Increase the robustness of cone detection against lighting. Current HSV thresholding is the largest single contributor to row-end-turning failure and a small, dedicated YOLO head for cone detection, trained on the same season of footage, would remove this sensitivity.
- Reduce mild box-loss overfitting by stronger augmentations and a longer training schedule to narrow the gap between training and validation box loss seen in Fig. 26.
- Swap GoPro for the RealSense colour stream as this would push the BoM below PKR 150,000 without measurable loss in detection quality and simplify the wiring/mount.
- Multiple RowSense units operating on parallel rows would cut total scouting time by a factor of N and naturally support cross-validation of detections.
- A 4G/LoRa link would stream a low-bandwidth telemetry feed (heading, lateral offset, detected counts) to the operator, useful for unsupervised overnight runs.
- Since the pipeline is crop-agnostic, recollecting a comparable seasonal dataset on rice, maize, or cotton would broaden the impact directly.

In conclusion, RowSense demonstrates that a low-cost, autonomous platform can deliver under-canopy crop scouting at a price point compatible with academic and small-farm budgets in Pakistan, while reaching detection performance on the most actionable agronomic class that

is competitive with much more expensive commercial offerings. The combination of an inexpensive chassis, a single depth camera, a single MEMS IMU, a Raspberry Pi 5, and a carefully trained YOLO model is sufficient to close the practical gap between UAV-scale monitoring and manual scouting.

REFERENCES

- [1] S. Savary, L. Willocquet, S. J. Pethybridge, P. Esker, N. McRoberts, and A. Nelson, 'The global burden of pathogens and pests on major food crops', *Nature Ecology & Evolution*, vol. 3, no. 3, pp. 430–439, 2019. [Online]. Available: <https://www.nature.com/articles/s41559-018-0793-y>. [Accessed: 11- Jan- 2026].
- [2] EarthSense, Inc. “TerraSentia™ Autonomous Under-Canopy Phenotyping Robot.” Available: <https://www.earthsense.co/terrasentia>. [Accessed: 11- Jan- 2026].
- [3] K. Kim, A. Deb, and D. J. Cappelleri, “P-AgBot: In-Row & Under-Canopy Agricultural Robot for Monitoring and Physical Sampling,” *IEEE Robotics and Automation Letters*, vol. 7, no. 3, pp. 7942–7949, Jul. 2022, doi: 10.1109/LRA.2022.3187275.
- [4] García-Navarrete, Oscar Leonardo, et al. “Application of Convolutional Neural Networks in Weed Detection and Identification: A Systematic Review.” *MDPI*, publisher, 2 Apr. 2024, www.mdpi.com/2077-0472/14/4/568.
- [5] I. N. Yulita, M. F. R. Rambe, A. Sholahuddin, and A. S. Prabuwono, “A convolutional neural network algorithm for pest detection using GoogleNet,” *MDPI*, <https://www.mdpi.com/2624-7402/5/4/145> [Accessed: 11- Jan- 2026].
- [6] M. S. Krishna et al., “Plant Leaf disease detection using Deep Learning: A multi-dataset approach,” *MDPI*, <https://www.mdpi.com/2571-8800/8/1/4> [Accessed: 11- Jan- 2026].
- [7] S. Islam et al., “Nutrient stress symptom detection in cucumber seedlings using segmented regression and a mask region-based convolutional neural network model,” *MDPI*, <https://www.mdpi.com/2077-0472/14/8/1390> [Accessed: 11- Jan- 2026].
- [8] A. R. Bahtiar, J. Juhariah, Pranowo, and A. J. Santoso, “Deep learning detected nutrient deficiency in chili plant,” *CORE*, <https://core.ac.uk/works/125169263/> [Accessed: 11- Jan- 2026].

- [9] Z. Li, Y. Li, Y. Yang, R. Guo, J. Yang, J. Yue, and Y. Wang, “A high-precision detection method of hydroponic lettuce seedlings status based on improved Faster R-CNN,” *Computers and Electronics in Agriculture*, vol. 182, Art. no. 106054, 2021, doi: 10.1016/j.compag.2021.106054.
- [10] A. N. Sivakumar, S. Modi, M. V. Gasparino, C. Ellis, A. E. B. Velasquez, G. Chowdhary, and S. Gupta, “Learned visual navigation for under-canopy agricultural robots,” in *Proceedings of Robotics: Science and Systems (RSS)*, Virtual Conference, USA, Jul. 2021, doi: 10.15607/RSS.2021.XVII.019.
- [11] Food and Agriculture Organization of the United Nations, “Understanding the Context – Pest and Pesticide Management.” [Online]. Available: <https://www.fao.org/pest-and-pesticide-management/about/understanding-the-context/en/>. [Accessed: 11- Jan- 2026].
- [12] DevelopmentAid, “Pests destroy up to 40 percent of global crops and cost \$220 billion of losses,” 2 Jun. 2021. [Online]. Available: <https://www.developmentaid.org/news-stream/post/92599/pests-destroy-global-crops>. [Accessed: 11- Jan- 2026].
- [13] CABI, “CABI scientists track wheat aphids and their natural enemies for better pest management in Pakistan,” 15 Oct. 2019. [Online]. Available: <https://www.cabi.org/news-article/cabi-scientists-track-wheat-aphids-and-their-natural-enemies-for-better-pest-management-in-pakistan/>. [Accessed: 11- Jan- 2026].
- [14] United Nations, “The 17 Goals – Sustainable Development.” [Online]. Available: <https://sdgs.un.org/goals>. [Accessed: 11- Jan- 2026].
- [15] Ultralytics, “Ultralytics YOLO11 – model documentation.” [Online]. Available: <https://app.vessl.ai/hub/yolo11>. [Accessed: 11- Jan- 2026].